



COPILOTTRAINING
TECHNICAL BRIEFING

From Issue to Merge Decision

| **Agentic workflows, Copilot review, and Code Quality together**

Bound the work · Inspect the draft · Enforce the bar · Keep acceptance human

CopilotTraining · Verify and Govern · 50 minutes

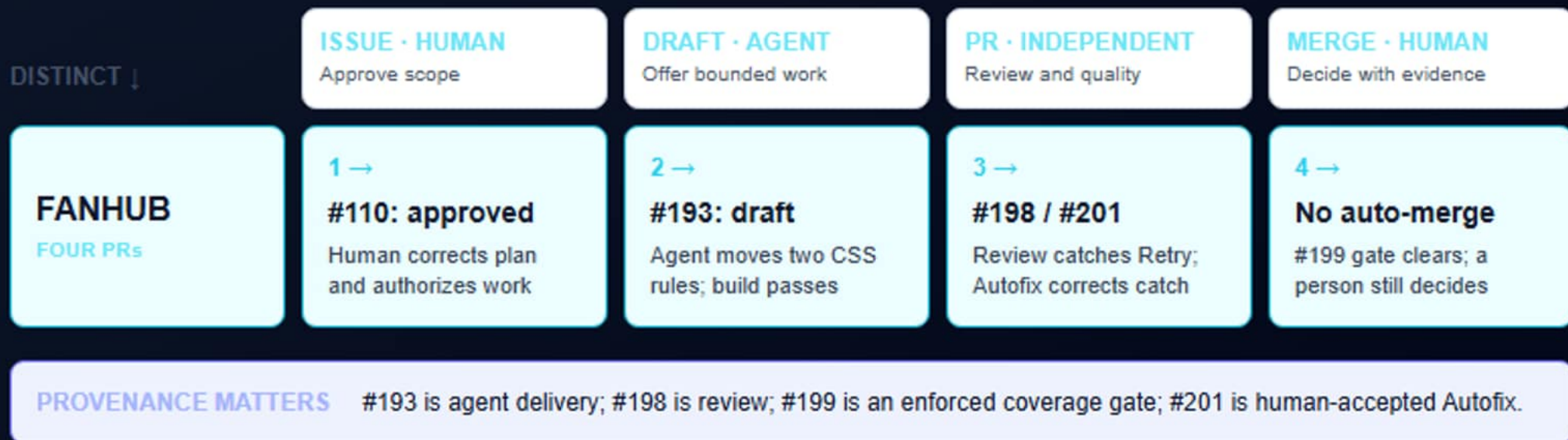
Who Acts, What Runs, and Who Decides?

An approved request, a draft, and independent signals make the merge decision inspectable.

DECISIONS ↓	ISSUE · HUMAN Approve scope	DRAFT · AGENT Offer bounded work	PR · INDEPENDENT Review and quality	MERGE · HUMAN Decide with evidence
WHO ACTORS	Developer Names the task and authority	Workflow agent Implements approved scope	Copilot + quality Advice, fix, or gate	Reviewer Accepts or holds the draft
WHAT HANDOFF	Runnable plan Behavior, tests, exclusions	Draft diff One offered change	Head-specific proof CI, review, quality gate	Disposition Accept, revise, or hold
WHERE EVIDENCE	Issue Approval and scope	Run + PR Actor, code, head SHA	Checks + threads Ruleset enforces the bar	PR decision Owner and recovery path

Four PRs Prove Different Parts of the Delivery Loop

FanHub examples are separate PRs; no single draft completed all four paths.

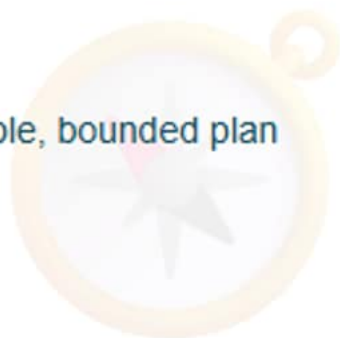


PART 1

Authorize the Work

Agentic lifecycle

Human approves a runnable, bounded plan



PART 2

Inspect the Draft

Agentic lifecycle

Read the actual diff, build, and residual risk

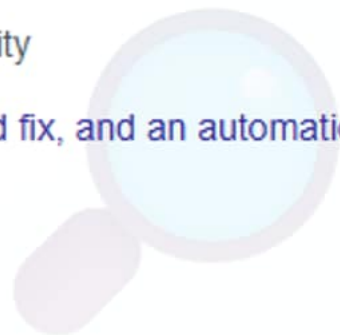


PART 3

Review and Enforce

Code review + Code Quality

Advice, a human-accepted fix, and an automatic gate



PART 4

Make the Merge Decision

Join the evidence

Know which PR proved what and what still needs a person



Authorize the Work

The lifecycle starts with a person choosing scope and a reproducible check.



Research

Issue #110 identifies layout work



Plan

Move only two CSS rules



Authorize

Human approves the corrected build path

FanHub issue #110 → draft PR #193

↳ Request labels hand off artifacts; status labels cannot approve a plan

A Failed Build Command Changes the Plan

FanHub #110 · revised plan

Request: move inline layout CSS
Scope: .main-content + .footer only

First command: dotnet build dotnet/FanHub.sln
→ fails on missing project paths

Corrected command:
dotnet build dotnet/Frontend/Frontend.csproj
→ 0 errors; 7 existing warnings

Actor

The maintainer tests the proposed command and asks for a corrected plan.

Evidence

A project build actually runs; existing warnings are recorded separately.

Boundary

A plan-ready label records output; the human approval event authorizes implementation.

The Human Approves Two Files and One Draft



Scope

MainLayout.razor + isolated CSS

Move only .main-content and
.footer

Preserve the visible layout



Proof

Build Frontend.csproj

Compare three routes at two
widths

Record what was not tested



Authority

Latest plan names @rbmathis

Maintainer checks the revised
plan

Approval label requests one draft



The agent can propose and implement; the named human decides when the plan is ready to run.

Which Actions Can GitHub Take Automatically?



Automated after setup

Lifecycle

Offer a draft after verified approval

Review

Comment on an eligible PR

Quality

Block merge when an active rule fails



Human decisions

Request

Approve scope and runnable checks

Correction

Inspect an Autofix before committing it

Acceptance

Review residual risk and decide on merge



These are different controls: Copilot advice is not a gate, and an Autofix suggestion is not an automatic commit.

PART 2

Inspect the Draft

The approved plan turns into an artifact a teammate can inspect.



Implement

Move the two approved CSS rules



Offer

One draft, not a merge



Verify

Read build and browser scope separately

FanHub issue #110 → draft PR #193

↳ Two-file CSS move · Frontend build passed · no automatic acceptance

The Approved Request Becomes a Draft, Not a Merge

gh-aw-implement-approved.md · source excerpt

```
on:
  issues:
    types: [labeled]
    names: [lifecycle:implement-approved]
safe-outputs:
  create-pull-request:
    title-prefix: "[lifecycle] "
    labels: [agent-generated, lifecycle:in-review]
    draft: true
    max: 1
```

Trusted approval

A pre-agent check verifies the label actor and latest approved plan.

Actual change

PR #193 moves two layout rules across MainLayout.razor and MainLayout.razor.css.

Output

The workflow offers a draft; compare the diff to the plan before any review.

A Passing Build and a Visible Layout Comparison

FanHub #193 · draft evidence

Changed: `MainLayout.razor`
`MainLayout.razor.css`

Project build: 0 errors
Warnings: 7 existing nullable warnings

Browser comparison: baseline vs draft
Routes: `/`, `/characters`, `/episodes`
Widths: 1024px and 300px
Main content + footer matched

Scope

The diff touches exactly the two approved layout files.

Domain result

Six baseline-to-draft viewport comparisons show the selected layout unchanged.

Limit

Blazor error recovery was not exercised; a build cannot settle that risk.

A Draft Can Be Useful While a Risk Remains Open



Runner

Bot PR CI initially required approval

Maintainer approved the read-only build

Build ran on the actual PR head



Artifact

Two-file diff matches the plan

Layout matches at tested widths

Error recovery is still unexercised



Decision

Ask for a focused runtime test

Accept residual risk under policy

Keep the draft while evidence is missing

PR #193 proves bounded delivery, not a merge. PR #198 separately shows why a passing test may miss browser behavior.

PART 3

Review, Remediate, and Enforce

Three distinct FanHub PRs reveal what advice, a fix proposal, and a gate can each do.



#198 · Review

Find the inert browser Retry



#199 · Gate

Block until coverage exceeds 27%



#201 · Autofix

Inspect and commit a generated fix

Separate draft PRs, not sequential heads of one PR

↳ Automatic review ≠ automatic remediation ≠ automatic merge gate

A Ruleset Requests Copilot Review on Draft #198

FanHub · active repository ruleset

Target: main
Automatic Copilot review: on
Draft pull requests: included
New pushes: included

PR #198: bot-created draft · ba0764f
Automatic review: COMMENTED
Two open inline findings

Setup

A maintainer enables draft and new-push triggers; a manual review request also works.

Output

Open the PR review and inline threads on the analyzed head, not just its check status.

Authority

Suggestions invite response and tests. COMMENTED is advisory, never a human approval.

Copilot Spots the Inert Retry Button

PR #198 · COMMENTED · 2 findings

Home.razor: button @onclick=LoadHomeDataAsync
Home: no interactive render mode
Routes.razor: <Routes /> (static)
→ click handler cannot run in browser

Retry also clears loadError first
→ alert and disabled state disappear

Agent reports 4/4 component tests pass

Actual review

Automatic Copilot review returned COMMENTED with two findings on the bot-created draft.

Why it matters

bUnit can invoke a click even when the real route renders only static HTML.

Next

Enable interactivity, keep the alert through Retry, then prove browser recovery.

A Failing Test and Browser Check Close the Gap



Wire interaction

Add InteractiveServer in the approved Home scope;
keep alert visible while Retry runs.



Test red → green

Render-mode test failed on ba0764f; 6/6 focused
tests pass locally on edeadee.



Reproduce failure

Mock API returns 503: browser shows alert and
server logs the exception.



Prove recovery

Mock returns 200; Retry restores count and quote,
clears alert, with one navigation total.

One Reliability Rule Finds a Silent Home-Page Failure

Initial Code Quality scan · main · 800c8ec

Rule: Poor error handling: empty catch block
Category: Reliability · severity: Note
File: dotnet/Frontend/ ... /Home.razor:332

```
catch { }  
→ failed API request goes unexplained
```

71 maintainability + 9 reliability
Existing default-branch findings

⚙️ Setup

A maintainer enabled Code Quality; rules-based CodeQL scanned main after activation.

🔍 Open live

Security and quality → Code quality → Standard findings; inspect the Home.razor rule and location.

🚩 Human response

Select issue #95, specify a friendly error and Retry, then verify the behavior.

The PR Scan Finds New Patterns to Triage

Rules-based PR scan · initial head

main · 800c8ec (existing)

1 empty catch in Home.razor

PR #198 · ba0764f (new)

1 generic catch in Home.razor

4 test HttpClient disposal findings

edeadee: specific catches + DI clients

New-head quality run: completed

🚩 Trigger and scope

Code Quality independently scanned the new PR head; the baseline finding stays on main.

📄 Observed output

Five inline findings identify a broad catch and four new test-client disposal patterns.

👤 Rerun

The edeadee scan completed successfully; inspect current findings, not only check status.

The Coverage Rule Blocks, Then Unblocks, the Same PR



First head · blocked

4bd7844

Six tests pass; Cobertura uploads

GitHub reports 26.7% below 27%

Active branch-scoped gate blocks merge



Human correction

ab96e8b

Author manually adds two Home tests

Eight tests pass on corrected head

These tests were not Autofix



Rule passes

PR stays draft

GitHub displays 30% coverage

Configured requirement now passes

Human acceptance still required

📌 github.com/MSBart2/FanHub/pull/199 · The 27% rule targets its isolated demo base, not PR #198 or #201.

An Author Accepts Code Quality Autofix

First head · 294529d

Generic catch clause

Episodes uses int.Parse plus catch (Exception)

Code Quality finding

Offers a generated Suggested changeset

Build passes

Yet unrelated exceptions can be swallowed

Autofix head · 0b33cda

Human clicks Commit suggestions

GitHub commits the generated TryParse fix

New-head checks

Build and C# analysis pass; finding is outdated

Limit

No frontend tests on main; PR remains draft

✦ github.com/MSBart2/FanHub/pull/201 · Copilot Code Review also commented, but Code Quality produced this Autofix. No gate is configured on #201.

Make the Merge Decision

The accepting reviewer joins current-head evidence without treating distinct PRs as one run.



Authority

Match plan, diff, and head



Proof

Distinguish advice, test, and gate



Decision

Accept, revise, or hold

Every FanHub example is still draft and unmerged
↳ Passing a check never supplies the human merge decision

Which PR Proves Which Part of the Story?



#193 · Lifecycle

Issue #110 approval authorizes a scoped CSS draft; build and layout compare pass.



#198 · Review

Copilot finds inert Retry after four reported tests pass; human tests the correction.



#199 · Gate

27% coverage rule blocks 26.7%; manually added tests take it to 30%.



#201 · Autofix

Code Quality proposes TryParse; the author commits it; scan and build rerun.

What Would Earn a Human Merge Decision?

Apply this checklist to one current PR head, not to evidence borrowed from other demos

Authority	Approved scope, event actor, and draft files match	SCOPE
Behavior	Relevant test or runtime proof exercises the change	PROOF
Review	Disposition of Copilot advice; re-review after fixes if needed	ADVICE
Quality	Inspect Autofix and gate; run separate security checks	RULE
Head	All required checks ran on the current commit	CI
Human	Accept residual risk or keep draft; deployment is next	DECIDES

Pilot the Connected Loop Before Expanding It

1

Authorize

Choose two or three bounded issues

Approve runnable plans and owners

Record what the agent may change

2

Observe

Inspect each draft and current head

Triage Copilot advice and quality findings

Prove one rule blocks and unblocks

3

Decide

Measure stops, rework, and reviewer time

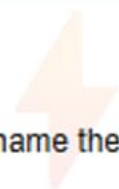
Keep human merge authority

Hand off deployment to its own controls

✓ 10–15 issues is a proposed intake target, not measured throughput. Deployment and production feedback are outside this demo.

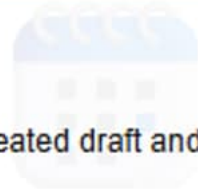
✓ WHAT YOU CAN DO TODAY

Today



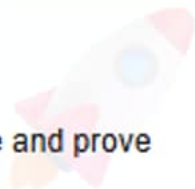
- Choose one issue and name the approver
- Correct the plan until its check runs
- Define what a draft may change

This Week



- Inspect one agent-created draft and head
- Request Copilot review and disposition
- Trial a Code Quality rule in evaluate mode

This Month



- Activate a tested gate and prove block → pass
- Inspect one Autofix before accepting it
- Define the post-merge deployment handoff

🔑 Key Takeaway

Automate the offer, advice, and enforceable bar; a human owns merge, then delivery continues.

Observed FanHub Artifacts

[Issue #110 · plan and approval](#)

Corrected build command and two-rule layout scope

[PR #193 · agentic draft](#)

Two-file layout fix, build, and browser comparison

[PR #198 · Copilot review](#)

Four green tests missed the inert browser Retry

[PR #199 · coverage gate](#)

26.7% blocked, then 30% after manually added tests

[PR #201 · Code Quality Autofix](#)

Generic catch finding, generated TryParse, author-accepted commit

Workflow and Signal Boundaries

[About Code Quality](#)

Default-branch and PR analysis have distinct scopes

[Configure Copilot review](#)

Draft and new-push settings

[Enable Code Quality](#)

Initial baseline scan and later activity

[Restrict code coverage](#)

Configured threshold plus required upload check

[Agentic safe outputs](#)

Bound the draft PR and advisory review



From Issue to Merge Decision

FOUR SEPARATE DRAFTS MAKE THE BOUNDARIES VISIBLE

#193

Human approval → agent-created draft

#198

Copilot advice → tested human correction

#199 / #201

Automatic gate / human-accepted Autofix

Who accepts your next AI-created PR, and what happens after merge?